Midterm Exam Preparation

Exercise 1

- Indicate whether the first function of each of the following pairs has a smaller, same, or larger order of growth (to within a constant multiple) than the second function.
  1. $n(n + 1)$ and $2000n^2$
  2. $100n^2$ and $0.01n^3$
  3. $\log_2 n$ and $\ln n$
  4. 2^{n-1} and 2^n
  5. $(n- 1)!$ and $n!$

Exercise 2

- Use the informal definitions of O, Ω ,Θ band to determine whether the following assertions are true or false.

a. $n(n+1)/2 \in O(n^3)$ **b.** $n(n+1)/2 \in O(n^2)$

c. $n(n+1)/2 \in \Theta(n^3)$ **d.** $n(n+1)/2 \in \Omega(n)$

TABLE 2.2 Basic asymptotic efficiency classes

Class	Name	Comments
1	*constant*	Short of best-case efficiencies, very few reasonable examples can be given since an algorithm's running time typically goes to infinity when its input size grows infinitely large.
$\log n$	*logarithmic*	Typically, a result of cutting a problem's size by a constant factor on each iteration of the algorithm (see Section 4.4). Note that a logarithmic algorithm cannot take into account all its input or even a fixed fraction of it: any algorithm that does so will have at least linear running time.
n	*linear*	Algorithms that scan a list of size n (e.g., sequential search) belong to this class.
$n \log n$	*linearithmic*	Many divide-and-conquer algorithms (see Chapter 5), including mergesort and quicksort in the average case, fall into this category.
n^2	*quadratic*	Typically, characterizes efficiency of algorithms with two embedded loops (see the next section). Elementary sorting algorithms and certain operations on $n \times n$ matrices are standard examples.
n^3	*cubic*	Typically, characterizes efficiency of algorithms with three embedded loops (see the next section). Several nontrivial algorithms from linear algebra fall into this class.
2^n	*exponential*	Typical for algorithms that generate all subsets of an n-element set. Often, the term "exponential" is used in a broader sense to include this and larger orders of growth as well.
$n!$	*factorial*	Typical for algorithms that generate all permutations of an n-element set.

Exercise 3

- List the following functions according to their order of growth from the lowest to the highest:

$$(n-2)!, \quad 5\lg(n+100)^{10}, \quad 2^{2n}, \quad 0.001n^4+3n^3+1, \quad \ln^2 n, \quad \sqrt[3]{n}, \quad 3^n.$$

Exercise 4

For each of the following algorithms, indicate(i) a natural size metric for its inputs, (ii) its basic operation, and (iii) whether the basic operation count can be different for inputs of the same size:

1. computing the sum of n numbers
2. computing n!
3. finding the largest element in a list of n numbers

Exercise 5

- For each of the following algorithms, indicate whether it is:
  - Stable?
  - In-place?
- Algorithms:
  - Insertion Sort
  - Selection Sort
  - Bubble Sort

Exercise 6

- Consider the following algorithm.

ALGORITHM *Mystery*(n)

```
//Input: A nonnegative integer n
S ← 0
for i ← 1 to n do
    S ← S + i * i
return S
```

1. What is its basic operation?
2. How many times is the basic operation executed?
3. What is the efficiency class of this algorithm?

Exercise 7

- Consider the following algorithm.

ALGORITHM $Secret(A[0..n-1])$

```
//Input: An array A[0..n − 1] of n real numbers
minval ← A[0]; maxval ← A[0]
for i ← 1 to n − 1 do
    if A[i] < minval
        minval ← A[i]
    if A[i] > maxval
        maxval ← A[i]
return maxval − minval
```

1. What is its basic operation?
2. How many times is the basic operation executed?
3. What is the efficiency class of this algorithm?

Exercise 8

- Consider the definition-based algorithm for adding two n × n matrices.
- What is its basic operation?
- How many times is it performed as a function of the matrix order n?
- As a function of the total number of elements in the input matrices?

```
Algorithm MatrixAddition(A[0..n-1], B[0..n-1])
  Input: Matrices A and B of size n × n
  Output: Matrix C of size n × n such that C[i][j] = A[i][j] + B[i][j]

  for i ← 0 to n - 1 do
    for j ← 0 to n - 1 do
      C[i][j] ← A[i][j] + B[i][j]

  return C
```

Exercise 9

- Given the algorithm below to finding the distance between the two closest elements in an array of numbers, answer the following questions.
  1. What's the basic operation?
  2. Write a summation that represents the number of times the basic operation is executed.
  3. Solve the summation and find the function which represents the number of times the basic operation is executed. Show your work.
  4. Classify the function above into one of the standard complexity classes using Θ-notation.

```
ALGORITHM  MinDistance(A[0..n − 1])
    //Input: Array A[0..n − 1] of numbers
    //Output: Minimum distance between two of its elements
    dmin ← ∞
        for i ← 0 to n − 1 do
            for j ← 0 to n − 1 do
                if i ≠ j and |A[i] − A[j]| < dmin
                dmin ← |A[i] − A[j]|
    return dmin
```

Exercise 10

- Given the following algorithm, answer the following questions.
  1. What's the basic operation?
  2. Write a summation that represents the largest number of times the basic operation is executed.
  3. Solve the summation and find the function which represents the largest number of times the basic operation is executed. Show your work.
  4. Classify the function above into one of the standard complexity classes using Θ-notation.

```
ALGORITHM FindFirstDuplicate(arr, size)
// Input: An array of integers, and its size
// Output: The first duplicate element found in the array, or -1 if no duplicates.

i <-- 0
while i < size do
   j <-- i + 1
   while j < size do
      if arr[i] = arr[j] then
         return arr[i]
      j <-- j + 1
   i <-- i + 1

return -1  // No duplicates found
```

Exercise 11

- Consider the following algorithm for sorting an array: for each element, it counts how many elements are smaller than it and uses this count to place the element in its correct position in the sorted array. Analyze the efficiency of this algorithm.

```
ALGORITHM  ComparisonCountingSort(A[0..n − 1])
    //Sorts an array by comparison counting
    //Input: Array A[0..n − 1] of orderable values
    //Output: Array S[0..n − 1] of A's elements sorted
    //    in nondecreasing order
    for i ← 0 to n − 1 do
        Count[i] ← 0
    for i ← 0 to n − 2 do
        for j ← i + 1 to n − 1 do
            if A[i] < A[j]
                Count[j] ← Count[j] + 1
            else Count[i] ← Count[i] + 1
    for i ← 0 to n − 1 do
        S[Count[i]] ← A[i]
    return S
```

Exercise 12

- Consider the following recursive algorithm.

```
Algorithm ComputeX(n)
    Input: Integer n ≥ 1 (assumed power of 2)
    Output: Value of x(n)

    if n == 1 then
        return c  // base case
    else
        return ComputeX(n / 2) + n
```

- What's the basic operation?
- Write a recurrence relation that represents the number of times the basic operation is executed.
- Solve the recurrence relation using backward substitution and find the function which represents the number of times the basic operation is executed. Show your work.
- Classify the function above into one of the standard complexity classes using Θ-notation.

Exercise 13

- Consider the following recursive algorithm.

```
Algorithm MysteryRecursive(n):
    if n <= 1:
        return 1
    else:
        return MysteryRecursive(n-1) + MysteryRecursive(n-1)
```

- What's the basic operation?
- Write a recurrence relation that represents the number of times the basic operation is executed.
- Solve the recurrence relation using backward substitution and find the function which represents the number of times the basic operation is executed. Show your work.
- Classify the function above into one of the standard complexity classes using Θ-notation.

Exercise 14

- Solve the following recurrence relations.

a. $x(n) = x(n-1) + 5$ for $n > 1$, $x(1) = 0$

b. $x(n) = 3x(n-1)$ for $n > 1$, $x(1) = 4$

c. $x(n) = x(n-1) + n$ for $n > 0$, $x(0) = 0$

d. $x(n) = x(n/2) + n$ for $n > 1$, $x(1) = 1$ (solve for $n = 2^k$)

e. $x(n) = x(n/3) + 1$ for $n > 1$, $x(1) = 1$ (solve for $n = 3^k$)